

Set 6 - UserAccount System (ArrayList-based)

Q1.

Create a UserAccount class with private fields for username, password, and balance. Add public getter and setter methods for balance only, and write a constructor to initialize username and password.

Answer:

```
public class UserAccount {
    private String username;
    private String password;
    private double balance;

    public UserAccount(String username, String password) {
        this.username = username;
        this.password = password;
        this.balance = 0.0;
    }

    // Getter and setter for balance only
    public double getBalance() { return balance; }
    public void setBalance(double balance) { this.balance = balance; }

    // Package-private getters used internally by login/registration logic
    String getUsername() { return username; }
    String getPassword() { return password; }

    public static void main(String[] args) {
        UserAccount user = new UserAccount("rahul_23", "pass123");
        user.setBalance(1000);
        System.out.println("Username: " + user.getUsername());
        System.out.println("Balance: Rs. " + user.getBalance());
    }
}

/* Sample Output:
Username: rahul_23
Balance: Rs. 1000.0
*/
```

Q2.

Write a Java program that uses an ArrayList to temporarily store multiple UserAccount objects. Implement a registration function that checks if the username already exists before adding a new user.

Answer:

```
import java.util.ArrayList;

class UserAccount {
    private String username;
    private String password;
    private double balance;

    public UserAccount(String username, String password) {
        this.username = username;
        this.password = password;
    }
}
```

```

        this.balance = 0.0;
    }

    public String getUsername() { return username; }
    public String getPassword() { return password; }
    public double getBalance() { return balance; }
    public void setBalance(double balance) { this.balance = balance; }
}

public class RegistrationDemo {
    static ArrayList<UserAccount> users = new ArrayList<>();

    public static boolean registerUser(String username, String password) {
        for (UserAccount u : users) {
            if (u.getUsername().equals(username)) {
                System.out.println("Registration failed: username '" + username + "' already exists.");
                return false;
            }
        }
        users.add(new UserAccount(username, password));
        System.out.println("Registration successful for username: " + username);
        return true;
    }

    public static void main(String[] args) {
        registerUser("rahul_23", "pass123");
        registerUser("priya_99", "mypwd");
        registerUser("rahul_23", "anotherpass"); // duplicate username
    }
}

/* Sample Output:
Registration successful for username: rahul_23
Registration successful for username: priya_99
Registration failed: username 'rahul_23' already exists.
*/

```

Q3.

Add a login function that takes a username and password, validates them against the ArrayList of UserAccount objects, and prints 'Login successful' or 'Invalid credentials' accordingly. Hint: Use a loop to search for the user and compare passwords using the getter method.

Answer:

```

import java.util.ArrayList;

class UserAccount {
    private String username;
    private String password;

    public UserAccount(String username, String password) {
        this.username = username;
        this.password = password;
    }

    public String getUsername() { return username; }
    public String getPassword() { return password; }
}

public class LoginDemo {
    static ArrayList<UserAccount> users = new ArrayList<>();

```

```

public static void registerUser(String username, String password) {
    users.add(new UserAccount(username, password));
}

public static void login(String username, String password) {
    for (UserAccount u : users) {
        if (u.getUsername().equals(username)) {
            if (u.getPassword().equals(password)) {
                System.out.println("Login successful");
            } else {
                System.out.println("Invalid credentials");
            }
        }
        return;
    }
    System.out.println("Invalid credentials"); // username not found
}

public static void main(String[] args) {
    registerUser("rahul_23", "pass123");

    login("rahul_23", "pass123"); // correct
    login("rahul_23", "wrongpass"); // wrong password
    login("unknown_user", "pass"); // no such user
}

/* Sample Output:
Login successful
Invalid credentials
Invalid credentials
*/

```

Q4.

Implement a method `addFunds(double amount)` in `UserAccount` that increases the balance and records a transaction string (like 'Added Rs.500') in a transaction history `ArrayList`. Display the updated balance and the latest transaction after each deposit.

Answer:

```

import java.util.ArrayList;

public class UserAccount {
    private String username;
    private double balance;
    private ArrayList<String> transactionHistory = new ArrayList<>();

    public UserAccount(String username) {
        this.username = username;
        this.balance = 0.0;
    }

    public void addFunds(double amount) {
        balance += amount;
        String record = "Added Rs." + amount;
        transactionHistory.add(record);
        System.out.println("Updated balance: Rs. " + balance);
        System.out.println("Latest transaction: " + record);
    }

    public static void main(String[] args) {

```

```

        UserAccount account = new UserAccount("neha_shah");
        account.addFunds(500);
        account.addFunds(250.50);
    }
}

```

```

/* Sample Output:
Updated balance: Rs. 500.0
Latest transaction: Added Rs.500.0
Updated balance: Rs. 750.5
Latest transaction: Added Rs.250.5
*/

```

Q5.

Build a `spendFunds(double amount)` method for `UserAccount` that deducts the amount only if the balance is sufficient, updates the transaction history, and prevents the balance from going negative. Constraint: Show an error message if the user tries to spend more than their balance.

Answer:

```

import java.util.ArrayList;

public class UserAccount {
    private String username;
    private double balance;
    private ArrayList<String> transactionHistory = new ArrayList<>();

    public UserAccount(String username, double balance) {
        this.username = username;
        this.balance = balance;
    }

    public void spendFunds(double amount) {
        if (amount > balance) {
            System.out.println("Error: Insufficient balance. Cannot spend Rs. " + amount +
                " (Available: Rs. " + balance + ")");
            return;
        }
        balance -= amount;
        String record = "Spent Rs." + amount;
        transactionHistory.add(record);
        System.out.println("Updated balance: Rs. " + balance);
        System.out.println("Latest transaction: " + record);
    }

    public static void main(String[] args) {
        UserAccount account = new UserAccount("aman_verma", 1000);
        account.spendFunds(300);    // valid
        account.spendFunds(900);    // exceeds remaining balance
    }
}

/* Sample Output:
Updated balance: Rs. 700.0
Latest transaction: Spent Rs.300.0
Error: Insufficient balance. Cannot spend Rs. 900.0 (Available: Rs. 700.0)
*/

```